YuvaIntern
Amanjeet

Week 4 Internship Report

Coding Style Guidelines & Inline Documentation

October 15, 2026

Internship and Job Opportunity Management System

This report proposes practical coding standards and inline documentation practices for the planned web-based system. It builds on earlier work covering project requirements, system overview, architecture, modules, and implementation planning.

Item	Details
Internship task	Week 4 — Coding Style Guidelines & Inline Documentation
Project	Internship & Job Opportunity Management System
Planned stack	Java, Spring Boot, HTML/CSS/JavaScript, MySQL, Git/GitHub
Approach	Student-level proposal with examples; no claim that the application has been implemented

Prepared as a proposed guideline document for consistent, readable, and maintainable project code.

Contents

Section	Topic
01	Introduction and connection to previous weeks
02	Coding style rules and naming conventions
03	Formatting and project file organization
04	Inline comments and API documentation
05	Examples from the proposed project
06	Tools and suggested workflow
07	Team collaboration, review checklist, and conclusion

1. Introduction

In the earlier internship reports, I worked on understanding the problem, identifying users and requirements, describing the proposed system architecture, and planning its modules and implementation roadmap. The next useful step is to decide how code should be written and documented before the project grows. These guidelines are a proposal for future development, not a claim that every rule has already been implemented.

The system is intended to help students find internship and job opportunities, submit applications, and track status updates. Recruiters or companies would manage listings and review applications, while administrators would maintain the platform. Since the project has separate modules—authentication, opportunity management, application management, notifications, and administration—consistent code will make those modules easier to understand and maintain.

Objectives of this report

- Define naming and formatting rules a student team can follow.
- Show how comments can explain business rules without repeating obvious code.
- Recommend documentation and formatting tools suited to the proposed technology stack.
- Suggest a simple review workflow for future development.

Guiding principle: Readable first, clever second. Code should be understandable to another student developer who has not written that module. Documentation should explain intent, assumptions, and important rules.

2. Coding Style Rules and Naming Conventions

Coding style is the set of conventions used to make source code consistent. The exact conventions can vary by language, but the team should agree on one approach and apply it across the project. The rules below are practical starting points for Java and the planned web technologies.

Area	Proposed rule	Example
Java classes	Use PascalCase; choose a noun that describes responsibility.	ApplicationService
Methods	Use lower camelCase and a clear action name.	findEligibleOpportunities()
Variables / fields	Use lower camelCase; avoid unclear abbreviations.	applicationStatus
Constants	Use uppercase words separated by underscores.	MAX_LOGIN_ATTEMPTS
Boolean values	Use names that read like true/false questions.	isEligible, hasApplied
HTML/CSS names	Use meaningful, consistent names; kebab-case for CSS classes.	.opportunity-card
Database columns	Use one consistent naming convention across the schema.	application_status
Files and classes	Keep file names aligned with the main class/component.	ApplicationService.java

2.1 Naming: unclear vs clear

```
// Less clear
int x = 30;
String s = "Applied";

// Clearer
int minimumEligibilityMarks = 30;
String applicationStatus = "Applied";
```

The second version takes slightly more space, but a reader can understand the meaning without searching elsewhere. Names should describe the purpose of a value, not just its data type.

2.2 Keep methods focused

A method should ideally perform one clearly defined task. For example, checking whether an opportunity is active should be separate from sending a notification. Smaller methods are easier to test, review, and document. If a method becomes difficult to explain in one or two sentences, it may be doing too many things.

2.3 Consistency matters more than personal preference

The team should not alternate randomly between **getApplicationStatus()**, **application_status()**, and **statusOfApplication()** for the same operation. Once a convention is selected, new code and code reviews should follow it.

3. Formatting and Project File Organization

Formatting does not change what a program is meant to do, but it changes how easily people can read it. Consistent indentation, spacing, and file organization reduce visual clutter and make mistakes easier to spot.

Rule	Recommended guideline	Reason
Indentation	Use four spaces in Java; avoid mixing tabs and spaces.	Makes block structure obvious.
Braces	Keep opening braces on the same line for Java methods and control blocks.	Creates predictable layout.
Blank lines	Separate logical sections; avoid unnecessary gaps.	Improves scanning.
Line length	Aim for around 100–120 characters where practical.	Reduces horizontal scrolling.
Imports	Remove unused imports and group them consistently.	Keeps file headers clean.
Error handling	Handle expected failures deliberately and use clear messages.	Helps debugging and user feedback.
Magic values	Use named constants for values with business meaning.	Makes rules easier to update.

3.1 Example: formatting makes logic easier to scan

```
// Hard to scan
if(isLoggedIn&&isActive&& !hasApplied){submitApplication();}

// More readable
if (isLoggedIn && isActive && !hasApplied) {
    submitApplication();
}
```

The improved version uses spaces around operators, consistent spacing, and a separate line for the action. In a real application, conditions should also be named or extracted if they become complicated.

3.2 Proposed high-level folder organization

```
internship-job-management/
  frontend/
    pages/
    styles/
    scripts/
  backend/
    src/main/java/.../
      auth/
      student/
      opportunity/
      application/
      notification/
      admin/
    src/main/resources/
  database/
    schema/
    sample-data/
  docs/
    requirements/
    design/
    coding-guidelines/
  README.md
```

This is a proposed structure, not a verified snapshot of an existing repository. The benefit is that a developer can quickly locate interface files, backend modules, database resources, and documentation.

4. Inline Comments and API Documentation

Comments are most useful when they explain **why** a decision was made, what a business rule means, or what assumptions a method relies on. Comments should not merely repeat the statement immediately below them. When code changes, its comments must also be reviewed so they do not become misleading.

4.1 Commenting rules

- Explain non-obvious business rules, edge cases, and important constraints.
- Use short, direct sentences and correct spelling.
- Document why a validation exists when the reason is not obvious.
- Do not write a comment for every line of straightforward code.
- Update or remove comments when the related logic changes.
- Never put passwords, access tokens, private user data, or secrets in comments.

4.2 Example: useful comment

```
// Prevent a second application for the same student and opportunity.
if (applicationRepository.existsByStudentIdAndOpportunityId(
        studentId, opportunityId)) {
    throw new DuplicateApplicationException();
}
```

This comment records the business rule behind the check. The method and exception names are illustrative examples for the planned Java/Spring Boot backend; they are not presented as code already implemented in the project.

4.3 JavaDoc for public methods

```
/**
 * Checks whether a student can apply for an opportunity.
 *
 * @param studentId the ID of the student
 * @param opportunityId the ID of the opportunity
 * @return true when the opportunity is open and no application exists
 */
public boolean canApply(long studentId, long opportunityId) {
    // Validation logic would be implemented here.
    return false;
}
```

JavaDoc uses a structured comment format that can be processed by documentation tools. This example describes the method's intended contract; the placeholder return value is not production logic.

4.4 Documentation types

Type	Best used for	Example
Inline comment	Explaining a non-obvious rule near code.	Why duplicate applications are blocked.
JavaDoc	Public classes/methods, parameters, return values, exceptions.	canApply(studentId, opportunityId).
README	Project purpose, setup instructions, folders, how to run tests.	Getting started with the project.
Design document	Architecture, module responsibilities, diagrams, decisions.	Frontend → Backend/API → MySQL.

5. Applying the Guidelines to the Proposed System

This section connects the rules to the system described in my previous reports. The examples are small design-level illustrations of how consistent naming and documentation could help when implementation starts.

5.1 Mini case study: application status

The system may use statuses such as Applied, Under Review, Shortlisted, Selected, and Rejected. A consistent set of status values helps avoid spelling variations such as “Short List” or “shortlisted” being treated as different values. The team should define allowed statuses in one place and document which role can update them.

Scenario	Guideline applied	Expected benefit
Student submits an application	Use names such as applicationStatus and opportunityId.	The data's meaning is clear.
Student tries to apply twice	Document the duplicate-application rule near validation.	Future developers understand the rule.
Recruiter updates status	Use approved status values and document allowed transitions.	Reduces inconsistent status handling.
Admin removes expired listing	Document expiry rules and access permissions.	Supports safer maintenance.

5.2 Application flow and where documentation helps

Figure 1. Proposed application flow. The validation step is a useful place to document important business rules.

5.3 Example: method names and responsibility

Illustrative method name	Responsibility
findOpenOpportunities(filters)	Finds listings matching filters that are still open.
canApply(studentId, opportunityId)	Checks conditions for submitting an application.
submitApplication(studentId, opportunityId)	Coordinates application submission.
updateApplicationStatus(applicationId, newStatus)	Updates status after permission and transition checks.
sendStatusNotification(applicationId)	Prepares a notification after a relevant status change.

These are proposed names for planning purposes, not a claim that the methods already exist.

5.4 Example documentation note

Business rule: A student should not submit more than one application for the same opportunity. The system should verify the existing application record before saving a new one. If an application already exists, the user should receive a clear message instead of a duplicate record.

6. Tools and Suggested Workflow

Tools can make the proposed guidelines easier to follow. A formatter can align code, but a developer still needs to decide whether a method is well designed and whether its documentation is accurate.

Tool	Role in workflow	How I would use it
Visual Studio Code / IntelliJ IDEA	Editor and language support.	Use formatting options, syntax highlighting, and code navigation.
Java formatter	Applies consistent Java layout.	Format changed files before review.
Checkstyle	Checks Java style rules.	Use shared settings to flag naming and formatting inconsistencies.
Javadoc	Generates API documentation from JavaDoc comments.	Generate reference docs for selected public classes/methods.
Git and GitHub	Version history and collaboration.	Commit focused changes and review before merging.
Markdown README	Basic project documentation.	Keep setup steps, module overview, and contribution rules accessible.

6.1 Suggested documentation workflow

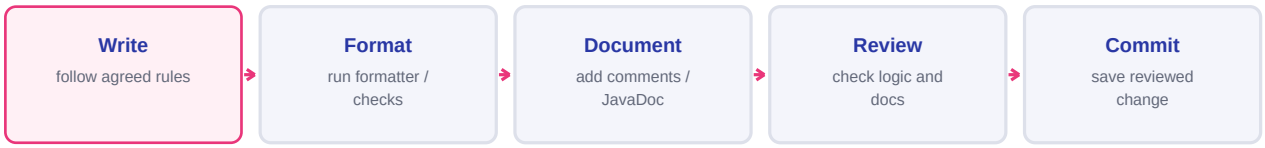

Figure 2. A lightweight workflow suitable for a student project.

6.2 Practical review checklist

- Are class, method, variable, and constant names consistent?
- Is indentation and spacing consistent with the agreed rules?
- Does each method have one clear responsibility?
- Are important business rules and edge cases documented?
- Do comments match the current code?
- Have debugging statements and unused imports been removed?
- Does the change avoid exposing credentials or personal data?
- Has the README or design note been updated if behavior or setup changed?

6.3 Keeping the workflow realistic

For a student project, I would start with a small set of rules and apply them consistently rather than adding too many tools at once. First, agree on naming and formatting. As the codebase grows, automated checks can reduce repeated manual work. Tools and configurations should be documented so every team member can use the same setup.

7. Consistency, Team Collaboration, and Conclusion

Coding standards are especially useful when more than one person works on the same project. If each developer uses different naming and formatting conventions, code reviews become harder because reviewers must first understand each person's style. A shared standard keeps attention on the actual logic, correctness, and security of a change.

Practice	How it supports the team
Shared naming and formatting rules	Makes code from different contributors look consistent.
Useful comments and JavaDoc	Helps a new contributor understand intent and method contracts.
Small, focused changes	Makes reviews easier and reduces unrelated changes.
Automated formatting / style checks	Catches repeatable style issues before or during review.
Updated README and design notes	Helps contributors set up the project and understand module boundaries.

7.1 Expected benefits

- Readability: developers can scan code and understand its purpose more quickly.
- Maintainability: future changes are easier when responsibilities and business rules are clear.
- Fewer avoidable mistakes: consistent names and documented constraints reduce misunderstandings.
- Better collaboration: reviews can focus on behavior rather than formatting preferences.
- Professional documentation: the project becomes easier to explain to another student or evaluator.

7.2 Limitations and next steps

These guidelines are a proposed baseline. They should be reviewed and adjusted when the final repository structure and implementation details are confirmed. A useful next step would be to add a short coding-guidelines section to the repository README, agree on formatter settings, and apply the checklist during future code reviews. Examples should be kept synchronized with the real implementation so the documentation remains accurate.

Conclusion

During Week 4, I prepared a proposed set of coding style and inline documentation guidelines for the Internship & Job Opportunity Management System. The report covers naming conventions, formatting, folder organization, useful comments, JavaDoc, practical examples, documentation tools, and a suggested review workflow. These practices can help keep the planned modules—authentication, opportunity management, application management, notifications, and administration—consistent as development progresses. As a student, I see this as an important step between planning the system and implementing it: readable code and clear documentation can make future debugging, teamwork, and maintenance much easier.

References / tools for further use

• Java documentation comments (JavaDoc) — official Java documentation.
• Checkstyle — Java source code style checker.
• GitHub documentation — repositories, commits, and pull requests.
• Visual Studio Code / IntelliJ IDEA — editor formatting and code navigation features.

Report note: examples and workflows are illustrative proposals for the planned system. They should not be interpreted as proof that application code or automated checks have already been implemented.